AAROHAN INTER-CENTRE HACKATHON 2026

AAROGYA DRISHTI

DETAILED OPEN-BOOK TECHNICAL & DEVELOPER HANDBOOK

Build • Integrate • Test • Secure • Deploy • Demonstrate

27 October – 4 November 2026

ITDA Paderu | Alluri Sitharama Raju District, Andhra Pradesh

Maximum 5 students per team | Winning team prize: ₹10,000

Hackathon demonstration only — NOT FOR CLINICAL USE

1. Purpose, boundaries and success criteria

This handbook is the common technical reference for every participating AAROHAN team. The hackathon is open-book: public documentation, AI coding assistants (including Google Antigravity), tutorials, licensed datasets and open-source libraries are permitted. Every team must independently understand, integrate, test and demonstrate its own system. The challenge is a genuine functioning product, not a collection of design screens.

Clinical safety boundary: The competition uses only synthetic patient records, simulated consultations and appropriately licensed educational photographs. Any student-built oral-image classifier is a research demonstration; it must not make clinical decisions, reassure patients or be used with real patient data. Real-world use needs health-department approval, qualified clinical supervision, independent security assessment and any required medical-device and research permissions.

What must work on the final day

- Registration with consent, Telugu/English form entry, clinical checklist and oral photograph capture.
- Image-quality checks and a clearly labelled mock or research AI pipeline that produces structured outputs.
- Medical Officer review independent of AI; specialist appointment and working two-party video demonstration.
- Referral, simulated biopsy/pathology report, diagnosis entered by clinician, follow-up and dashboard updates.
- Protected accounts and patient-level access, persistent database, audit trail and offline-to-online recovery.
- Application accessible on a real Android phone and deployed test URL with documented test accounts.

2. Nine-day delivery strategy and team allocation

Role	Core responsibilities	Definition of done
Engineering lead / backend	Git, FastAPI, database, schema, API, integration	Working secure end-to-end APIs
Mobile/frontend engineer	PWA, camera, offline, bilingual forms	Screening flows on real Android phone
AI/integration engineer	OpenCV quality, mock adapter, optional research model	Reproducible AI response; no medical claims
Clinical workflow/frontend	Doctor portal, referral, video, navigator, dashboard	Complete simulation journey
QA/security/DevOps	Automated tests, authorization, deployment, monitoring	Negative tests, secure deployment and runbook

Teams of fewer than five must combine roles. Agree on a single interface contract before working in parallel. Hold a 15-minute daily stand-up and integrate code twice per day. The minimum complete system should be demonstrable by 31 October; the remaining days are for functionality and hardening.

Date	Minimum milestone	Evidence
27 Oct	Read brief, assign roles, user journeys, model risk register	PRD, architecture, repo
28 Oct	Scaffold app/API/DB; login + authorization	Running local stack

Date	Minimum milestone	Evidence
29 Oct	Registration, consent, Telugu strings, clinical intake	Saved patient and visit
30 Oct	Photographs, quality checks, versioned AI adapter	Image->AI->persisted result
31 Oct	Medical Officer review, signed note	Independent review and referral
1 Nov	Video call, booking, appointment status	Two accounts join call
2 Nov	Diagnostic tracking, navigator, district dashboard	End-to-end walkthrough
3 Nov	Security, edge cases, deployment, repeatable tests	Passed/failed test report
4 Nov	Final demo, repository, video, documentation	Jury can independently use app

3. System architecture and folder structure

Suggested practical stack: React + TypeScript + Vite PWA, FastAPI/Python, PostgreSQL, private object storage, OpenCV, pluggable AI adapter, and approved demo video SDK. Teams may choose alternatives if the functionality and security are demonstrated. A PWA does not automatically guarantee background sync on every Android/browser combination: test on real devices.

```
aarogya-drishti/
  apps/web/              # PWA + health worker, clinician, admin UI
  services/api/          # FastAPI, authorization, clinical workflow
  services/ai/           # image quality + mock/research adapters
  infra/                 # Docker and deployment manifests
  docs/                  # architecture, permissions, API, tests
  tests/                 # integration + E2E + security tests
  .env.example           # placeholders ONLY; never secrets
  README.md
```

Core data flow

Health worker → signed-in PWA → authenticated FastAPI → database + private images → AI adapter (research only) → stored AI observation → Medical Officer → video or physical specialist → diagnostics → navigator → aggregate dashboard. The clinical workflow must continue if the AI service is unavailable.

4. Start development in Antigravity

Antigravity is an AI-assisted coding environment. Each team should work in a Git-controlled folder, request a design before implementation, inspect all generated code, run tests and commit accepted changes. AI assistant suggestions are not proof of correctness. Never paste real medical records, passwords, secrets or identifiable photographs into prompts.

Master planning prompt — paste into Antigravity

```
Build "Aarogya Drishti" as a SYNTHETIC-DATA-ONLY hackathon demo.

Stack: React+TypeScript PWA, FastAPI, PostgreSQL, private image store,
```

```
OpenCV image-quality checker, mock/research AI adapter, Docker.
Roles: health worker, medical officer, specialist, navigator, admin.
Workflows: consent, registration, screening, photos, AI output,
independent clinician review, secure video appointment, referral,
biopsy/pathology simulation, navigation, aggregate district dashboard.
Languages: Telugu and English. Offline draft + safe retry on Android.
First produce PRD, architecture, schema, permission matrix, API spec,
threat model, tests and a 9-day task breakdown. Then implement vertical
slices with passing tests. Never describe research AI as clinical.
Ask for review before destructive migrations or changing security rules.
```

Prompt sequence for smaller increments

Stage	Example agent instruction
1 — foundation	Create Docker dev stack, migrations, health check, seed demo data and /docs API documentation.
2 — screening	Build Telugu/English registration, consent gates, camera with retake, patient duplicate warning.
3 — AI	Add a versioned mock adapter + OpenCV quality tests + stored AI output and nonclinical warnings.
4 — review	Build role-scoped work queue, clinician observations, signed review and referral action.
5 — continuity	Build appointments, video link, diagnostics, missed-visit navigator and dashboard.
6 — hardening	Generate tests for every role boundary, offline sync failure, secret leakage, backup and recovery.

5. Development workstation and local setup

Prerequisites: Git, Node.js LTS, Python 3.11+, Docker Desktop/Engine with Compose, Android Chrome, and an editor/Antigravity. Versions should be locked in the repository. Teams should confirm host operating-system requirements before using the commands below.

```
git init aarogya-drishti && cd aarogya-drishti
mkdir -p apps/web services/api services/ai docs tests infra
# Frontend
cd apps && npm create vite@latest web -- --template react-ts
cd web && npm install && cd ../..
# Backend
cd services/api && python -m venv .venv
# activate: Linux/macOS source .venv/bin/activate
# Windows: .venv\Scripts\activate
pip install fastapi 'uvicorn[standard]' sqlalchemy alembic
pip install psycopg[binary] pydantic-settings python-multipart
```

```
# Further dependencies MUST be pinned and reviewed.
```

Create .env.example for DATABASE_URL, object-storage credentials, JWT configuration and video SDK keys. Actual .env files and credentials must be gitignored. Use a password manager or deployment secret manager, and restrict access to deployment users.

6. Data model: entities, relationships and invariants

Entity	Minimum fields	Controls
users	id, role, assigned_facility, status, auth_id	Unique identity and active status
patients	id, demographic fields, residence, contact, duplicate_hint	PII limited to authorized users
consents	id, patient_id, purpose, language, timestamp, version	Immutable consent history
screenings	id, patient_id, examiner_id, date, symptoms, tobacco, exam	Clinician findings distinct from AI
images	id, screening_id, private_key, checksum, view, quality	Private blob, authorization per record
ai_analyses	id, image_id, model_version, research_label, status	Always nonclinical in hackathon
clinical_reviews	id, screening_id, doctor_id, finding, signed_at	Only clinician can finalize
referrals	id, review_id, facility, specialty, priority, status	Explicit clinical decision
appointments	id, referral_id, slot, status, video_ref	Status history + no public meeting links
investigations	id, referral_id, kind, state, report_key	Pathology is distinct from image label
navigation_tasks	id, referral_id, owner, due, status	Overdue escalation and closure
audit_events	actor, event, resource, time, metadata	Append-only; avoid PHI in logs

Data integrity: Prefer UUIDs, foreign-key constraints, validation at the API boundary, explicit status transition rules and soft-delete where policy demands retained records. Reject cross-patient linkage and silent overwrites. Only clinician-authorized events can mark a case as confirmed cancer in a simulated workflow.

Proposed state machine

DRAFT → CONSENTED → SCREENED → AWAITING_CLINICIAN → REVIEWED → REFERRAL_PLANNED → APPOINTED → ASSESSED → DIAGNOSTICS_PENDING → RESULT_REVIEWED → CARE_LINKED or CLOSED. Support RESCHEDULED, MISSED, DECLINED and LOST_CONTACT as tracked exceptions. Never let an AI result bypass clinician review.

7. API contract: minimum endpoints

Method + route	Purpose	Who may use
POST /auth/login	Create authenticated session	Permitted users
POST /patients	Register synthetic patient	Health worker
POST /patients/{id}/consents	Record purpose-specific consent	Health worker
POST /screenings	Create screening and history	Health worker
POST /screenings/{id}/images	Upload protected oral photo	Assigned health worker
POST /ai/analyze	Queue nonclinical image analysis	Authorized screening user
POST /reviews	Save clinician observations	Medical Officer
POST /referrals	Create specialist referral	Medical Officer
POST /appointments	Book or request appointment	Clinician/navigator
POST /investigations	Record biopsy/pathology simulation	Authorized clinician
PATCH /navigation/{id}	Track contact and attendance	Navigator
GET /dashboard/summary	Aggregate reporting only	District administrator

All endpoints must authenticate each request and authorize access to the particular patient/facility record, not merely check a role. Add request/response validation, pagination, error codes, idempotency for retryable writes, and testable API examples.

AI response — strictly research demo


```
{
 "analysis_id":"DEMO-AI-009", "screening_id":"DEMO-CASE-001",
 "image_quality":"acceptable", "provider":"mock",
 "model_version":"mock-v1", "research_label":"suspicious_example",
 "score":null, "clinical_use":false,
 "requires_clinician_review":true,
 "warning":"NOT FOR CLINICAL USE"
}
```


8. Image capture and quality-control engineering

- Build guided oral views: cheek lining, tongue, palate, gums and floor of mouth as clinically indicated. Display plain-language instructions and allow recapture.
- Validate file extension and decoded format, restrict size/dimensions, normalize orientation, strip unnecessary metadata and generate a checksum.
- OpenCV may flag blurry, underexposed, overexposed or glare-heavy images. It cannot ensure that a lesion is absent.
- Use private storage keys and short-lived signed access authorized by the backend. Never embed raw object-store access keys in the PWA.

- Store images only after consent gates and associate each with the correct screening ID. Reassignment across patients must fail.

Sample image-quality pseudocode

```
def check_quality(image):
    if image is None or image.width < MIN_WIDTH:
        return {"status": "recapture", "reason": "invalid_image"}
    gray = to_grayscale(image)
    blur = variance_of_laplacian(gray)
    exposure = mean_brightness(gray)
    # thresholds must be calibrated on demo images
    if blur < BLUR_THRESHOLD:
        return {"status": "recapture", "reason": "blurry"}
    return {"status": "acceptable", "reason": None}
```

9. AI model laboratory: dataset to demonstration service

The publicly accessible oral lesion research dataset discussed for the hackathon contains original benign and malignant lesion photographs and augmented variants. Check the downloaded files, label definitions and licence yourself. The model trained on these images is not automatically valid for screening healthy mouths, precancer or lesions from other cameras/communities.

Correct training workflow

- Download licensed dataset; write DATASET_CARD.md: origin, labels, counts, restrictions, image modalities, missing demographics, permitted uses.
- Remove corrupt files and inspect repeated subjects. Split by patient where IDs exist; otherwise group source originals and all related augmentations.
- Split original sources into train, validation and held-out test before augmentation. Do not optimize using the final test set.
- Use torchvision EfficientNet-B0 pretrained weights, replace classifier head with two research categories, fine-tune using only the training set.
- Report sensitivity, specificity, precision, F1, confusion matrix, uncertainty, prevalence limitations and actual test sample sizes.
- Use calibrated language: a model score is not the chance a patient has cancer. No production inference on patients.

Minimal model initialization example

```
from torchvision import models
from torch import nn
model = models.efficientnet_b0(
    weights=models.EfficientNet_B0_Weights.DEFAULT)
in_features = model.classifier[1].in_features
model.classifier[1] = nn.Linear(in_features, 2)
# Follow with a proper training + evaluation pipeline.
```

```
# Save: model weights, exact transforms, label map, dataset card,
# performance report, code revision and model version.
```

Localization warning: Do not show bounding boxes or heatmaps implying medical localization unless a properly supervised localization model exists and its limitations are documented. Class activation maps are not necessarily medically reliable lesion annotations. The safest default is an image-quality checker plus a simulated or distinctly research-labelled classifier.

10. Telugu, low literacy and offline-first design

- Use a stable translation dictionary: key → English/Telugu string. Do not hardcode English text into components.
- Test Telugu rendering on Android devices and PDF output. Use supported fonts and verify special characters, numeric entry and line wrapping.
- Use 44px-or-larger targets, clear progress steps, minimal required typing and visible save confirmation.
- Store offline drafts in an encrypted or suitably protected device store. A browser PWA offers limited protections and must be assessed; do not claim mobile-device encryption automatically protects application caches.
- Queue writes with idempotency keys and sync status: pending, uploading, confirmed, failed. Do not mark a case synced before server acknowledgement.
- On conflict (same case edited on two devices), show a resolution state; never silently discard clinician notes.

Offline failure sequence for jury

Switch phone to airplane mode → create synthetic patient and save screening draft → close and reopen app → verify draft remains → reconnect → sync once → force network retry → verify exactly one server screening record and correct timestamps.

11. Telemedicine, appointment and clinical-review workflows

Video call support should connect two test accounts using a secure demo SDK, authenticated appointment access and time-limited meeting tokens. No consultation recording by default. A teleconsultation is not a substitute for specialist examination/biopsy when clinically needed. Synthetic records only during competition.

Actor	Allowed actions	Restricted actions
Health worker	Register, screening intake, image capture, assigned follow-up	Cannot confirm cancer, view unrelated case
Medical Officer	Review clinical findings, decide clinical referral	Cannot silently rewrite another signed review
Specialist	View referred cases, consultation, simulated investigation reports	Cannot browse all district patients
Navigator	Appointments, contacts, transport and missed-visit workflow	Cannot access unnecessary images or change diagnosis
District admin	Aggregated screening/referral trends	No default access to identifiable patient records/images

The platform must record whether appointments were REQUESTED, CONFIRMED, COMPLETED, MISSED, CANCELLED or RESCHEDULED. Store the clinician outcome separately from attendance. Follow-up rules should be configurable under a clinical protocol rather than invented by the software.

12. Security and privacy implementation

- Authentication: hashed passwords using a modern password algorithm, session expiry, secure cookies or safely handled bearer tokens, MFA for privileged users.
- Authorization: enforce actor role AND patient-level/facility-level access on backend queries; deny-by-default.
- Transport + storage: HTTPS, encrypted database/backups, private object storage, credential rotation and minimum permission service accounts.
- Uploaded files: decode and inspect image formats, size limit, malware inspection where supported, safe filenames and private access.
- Audit every image view/export, clinical edit, diagnosis entry and permission change without copying sensitive content into logs.
- Separate dev/demo/prod; prohibit actual patient records during competition. Publish a demo data reset script.
- Threat model: stolen phone, lost token, wrong-record access, insecure direct object reference, leaked AI service key, malicious uploads, video link sharing, database backup leak.
- No hardcoded secrets, public image buckets, third-party tracking pixels, or sensitive details in notifications.

Security test prompt for Antigravity

```
Act as an application-security reviewer. For each API, try another
user's patient ID, forged role, expired token, absent consent, replayed
submission, oversized image, wrong MIME, traversal filename, mass
assignment, leaked signed URL and invalid status transition.
Create automated failing tests first, fix code, rerun, and present
real command output. Never mark a test passed without execution.
Use synthetic data only.
```

13. Comprehensive positive, negative and adversarial test plan

Test	Input / injection	Expected behavior
Normal journey	Consent → screening → photo → review → video → biopsy demo	Correct single longitudinal record
Blurred photo	Defocused image	Recapture warning; no reliable inference claim
Invalid content	Random PDF renamed .jpg	Reject after file decoding
Missing consent	Attempt image upload before consent	Reject; record explanation
Wrong patient ID	Clinician A requests unrelated case	403 or equivalent access denied
AI down	Kill inference container	Case intact; manual clinical review available
Network lost	Disconnect during image upload	Retryable state; no duplicate image
Double submit	Send same POST twice with same key	Single durable record
False-negative AI	Model says low concern; clinician suspicious	Referral action remains available
False-positive AI	Model says suspicious; clinician disagrees	Clinician may document discrepancy
Missed appointment	Mark appointment missed	Follow-up task created
Broken video link	Use expired/wrong meeting token	Deny joining; allow reschedule
Long Telugu strings	Change language, PDF and mobile view	Readable without overflow
Concurrent edits	Two users update same review	Conflict or explicit version handling
Dashboard totals	Seed known 12 cases, 4 referrals	Exact aggregate count
Audit trail	Change pathology status	Actor/time/action preserved

Required evidence: unit tests, API integration tests, browser/mobile end-to-end tests, authorization tests, offline demonstrations and a known-issues list. Judging should include tests not disclosed in advance. Test datasets must remain synthetic.

14. Deployment runbook and recovery

Minimum deployment arrangement

Layer	Demo-ready option	Must verify
PWA	HTTPS static hosting or web container	Mobile camera, reload and offline
API	Managed container/VM	TLS, secrets, health endpoint
PostgreSQL	Managed DB or locked down instance	Migrations, backup/restore
Images	Private object storage	No public listing/download
AI	Optional mock/research container	Can fail without app failure
Video	SDK/provider under demo account	Secure links and two-device call
Monitoring	Structured logs + uptime checks	No PHI or secrets in logs

Example deployment checks

```
docker compose up --build -d

# Run Alembic migrations and seed synthetic demo data

# Verify API: GET /healthz returns 200

# Verify frontend URL on Android with HTTPS

# Verify database survives API restart

# Verify object storage is private, token expires

# Run automated tests against deployed staging URL

# Disable test users and rotate secrets after event
```

Free-tier services may sleep, change quotas or lack persistent filesystems. Check current offerings and avoid relying on a service that deletes uploads after a restart. Students must submit a second reproducible local setup route if live hosting fails.

Incident and recovery rehearsal

Take a database backup and restore into a fresh demo environment. Simulate an unavailable AI service, expired video link and failed photo upload. Demonstrate that the screening record remains intact, events are logged, and retries do not duplicate cases.

15. Documentation and final submission package

- README: installation, architecture, chosen stack, testing credentials, synthetic demonstration steps and known limitations.
- Architecture diagram and data flow with trust boundaries; ER diagram; role-access matrix.
- OpenAPI documentation with example requests/responses; environment-variable table without secrets.
- AI model card and dataset card, including licences, train/test separation, failures and nonclinical disclaimer.
- Threat model, test report (pass/fail commands/screenshots), deployment and rollback/backup instructions.
- Git repository, functional demo URL, Android install/PWA instructions, 5-minute video and live 10-minute journey.

- Contribution record and disclosure of AI coding tools, third-party templates and any external assistance.

16. Jury evaluation: 100 points + prize

Criterion	Marks
End-to-end patient workflow and integration	20
AI engineering, image quality and safe clinical integration	15
Medical Officer review, video and appointments	15
Records, diagnostics and patient navigation	15
Security, privacy, access restrictions	15
Telugu accessibility and offline reliability	10
Deployment, testing, maintainability and documentation	10
TOTAL	100

Prize: ₹10,000 to the winning team (not per student), subject to mandatory requirements. Team size: maximum five currently enrolled AAROHAN students; one team per student. Jury may require any member to explain implementation. An incomplete or unsafe demonstration cannot win solely on claimed model accuracy. Jury decision is final subject to approved rules.

17. Open-book competition policy and integrity

- Permitted: Antigravity, ChatGPT, Gemini, Copilot, documentation, tutorials, open-source components, openly licensed datasets and research papers.
- Disclose material reused code, licences and AI-assisted work; ensure students can demonstrate and explain the resulting code.
- Students do implementation, integration and presentation; mentors may guide and review but not submit their own completed solution as student work.
- Prohibited: collecting real patient photographs/medical details, presenting a research classifier as clinical, unapproved live telemedicine, hardcoded live service secrets, copying undisclosed proprietary implementations.
- Reproducibility matters: every team supplies a reset command, demo accounts and a path for jury deployment.

18. Common learning resources and official references

Topic	Resource
Antigravity	https://antigravity.google/
React	https://react.dev/learn
Vite	https://vite.dev/guide/
FastAPI	https://fastapi.tiangolo.com/tutorial/
PostgreSQL	https://www.postgresql.org/docs/
PyTorch	https://docs.pytorch.org/
OpenCV	https://docs.opencv.org/4.x/
Dataset (verify licence)	https://data.mendeley.com/datasets/mhjyrn35p4/2
OWASP API Top 10	https://owasp.org/API-Security/
IISc TANUH context	https://www.tanuh.ai/solutions/oral-cancer

Reference links are learning aids, not endorsements, automatic model access rights or confirmation that a medical AI API is publicly licensed for use.

19. Final live jury script

1) Open the deployed app in Telugu on a phone. 2) Register a synthetic 46-year-old resident of G. Madugula. 3) Record tobacco history, consent and a sample abnormal oral finding. 4) Photograph or upload licensed demo media; repeat with a blurry image. 5) Run the nonclinical AI adapter; deliberately stop it and show graceful recovery. 6) Log in as Medical Officer, review the case, disagree with AI if relevant and record a clinical decision. 7) Book video consultation and connect with a second test account. 8) Generate a physical referral, mark attendance and upload a simulated pathology report. 9) Update navigation and show correct dashboard counts. 10) Try unauthorized access and offline synchronization. 11) Explain limitations, data safety and what would be required before any real-world patient use.

The central goal: Deliver a secure, intelligible, functioning screening-to-care digital platform. A capable team must understand both what software can do and what only qualified clinicians can decide.